SZABIST
University

Shaheed Zulfikar Ali Bhutto
Institute of Science and Technology

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks:_____

Obtained Marks: _____

Computing Vision

Lab Task # 05

Submitted To: **Mr. Muhammad Azhar**

Student Name: **Habiba Bibi**

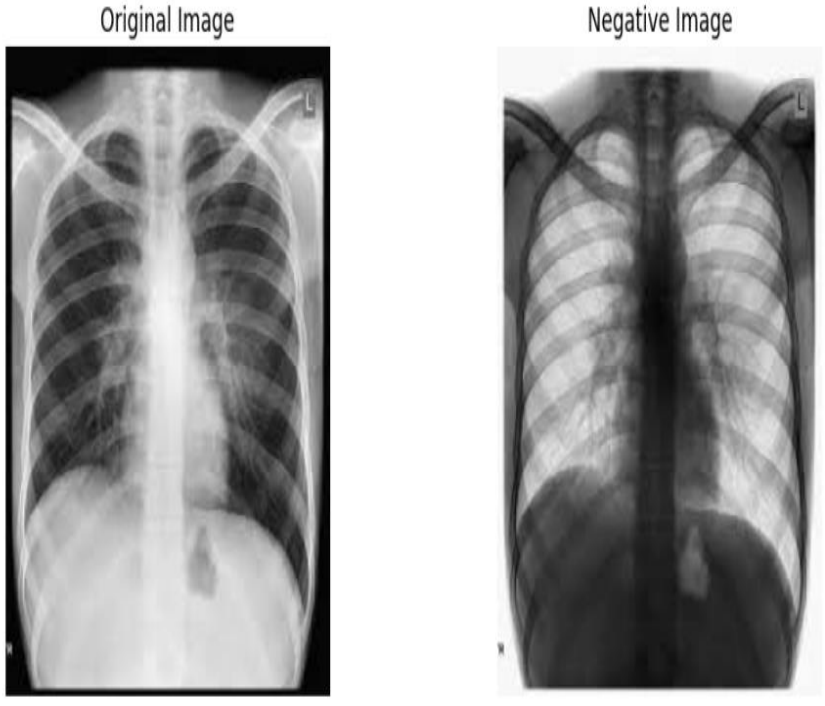
Reg. Number: **23108111**

Code Example 1: Image Negative

You are working with a medical research team at a hospital, helping them process chest X-ray images from the NIH Chest X-ray dataset, a publicly available medical imaging dataset used to train diagnostic AI models for conditions like pneumonia, tuberculosis, and lung nodules.

Some machine learning models and diagnostic tools respond better to image negatives, especially when abnormalities (like lesions) have inverse brightness characteristics. Your task is to create a preprocessing tool that performs negative transformation on grayscale chest X-ray images.

```
# Code Example 1: Image Negative
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108111/H12.jpg", cv2.IMREAD_GRAYSCALE)
negative = 255 - img
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(negative, cmap='gray')
plt.title("Negative Image")
plt.axis('off')
plt.tight_layout()
plt.show()
```

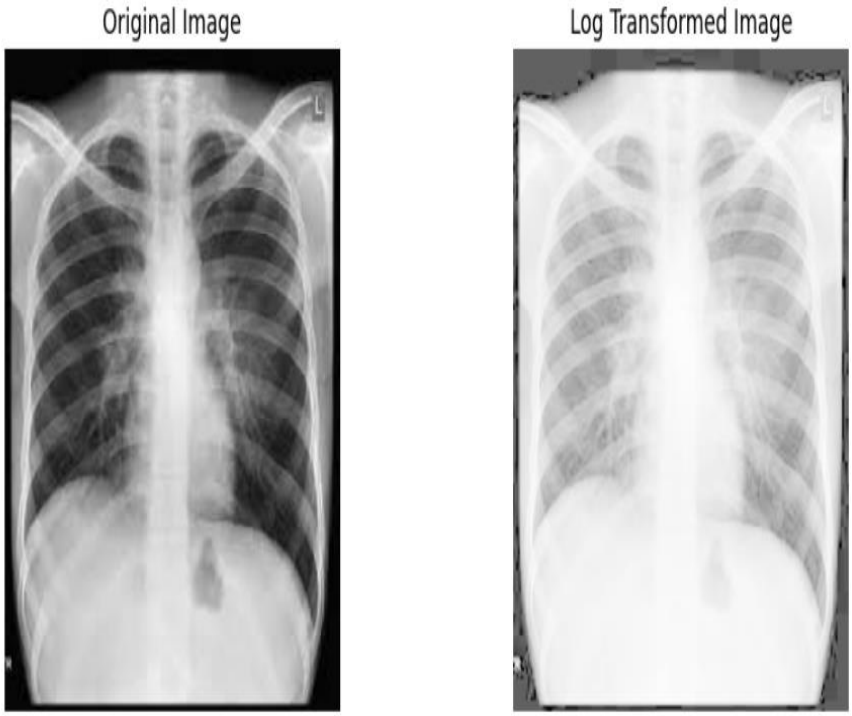


Code Example 2: Log Transformation

You are collaborating with a medical AI team that is analyzing the NIH Chest X-ray dataset for early detection of lung abnormalities such as pneumonia, fibrosis, or tuberculosis. The radiologists report that some critical structures (like minor opacities or nodules) are not clearly visible in raw X-ray images due to low contrast in the darker pixel regions.

They have requested your help in preprocessing these images using logarithmic transformation, a method known to enhance dark pixel values more than bright ones, making faint features more visible in diagnostic scans.

```
]: # Code Example 2: Log Transformation
img = cv2.imread("D:/23108111/H12.jpg", cv2.IMREAD_GRAYSCALE)
img_float = np.float32(img)
log_transformed = cv2.normalize(np.log1p(img_float), None, 0, 255, cv2.NORM_MINMAX)
log_transformed = np.uint8(log_transformed)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(log_transformed, cmap='gray')
plt.title("Log Transformed Image")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Code Example 3: Gamma Correction

You are working with a team of engineers analyzing thermal infrared images captured by drones for industrial inspection of solar panels and electrical grids. These thermal images often have low contrast, especially when detecting minor temperature variations across surfaces.

Due to varying ambient light and reflective surfaces, thermal hotspots sometimes appear too dark, making early faults in solar panels or electrical equipment difficult to detect. Your task is to use gamma correction to normalize the contrast of these images so that subtle temperature differences become more visible for both visual inspection and AI-based defect detection.

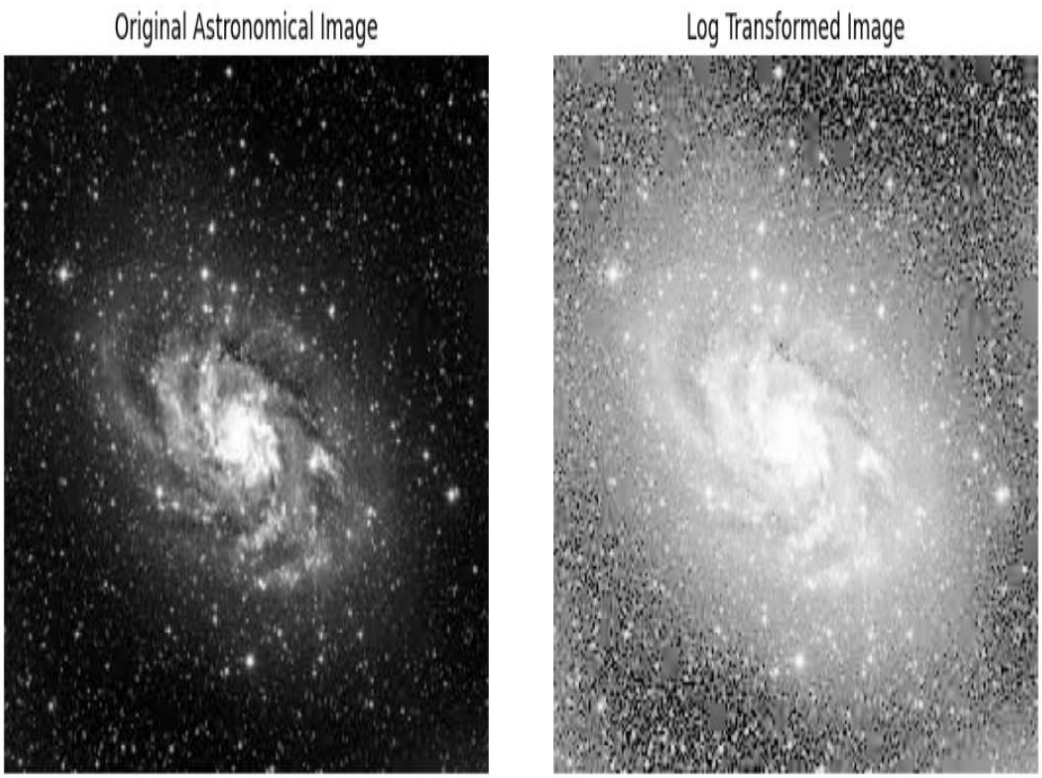
```
]: # Code Example 3: Gamma Correction
   img = cv2.imread("D:/23108111/H13.jpg")
   gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
   gamma = 0.4
   gamma_corrected = np.array(255 * (gray / 255) ** gamma, dtype='uint8')
   plt.figure(figsize=(10, 4))
   plt.subplot(1, 2, 1)
   plt.imshow(gray, cmap='gray')
   plt.title("Original Image")
   plt.axis('off')
   plt.subplot(1, 2, 2)
   plt.imshow(gamma_corrected, cmap='gray')
   plt.title(f"Gamma Corrected (γ = {gamma})")
   plt.axis('off')
   plt.tight_layout()
   plt.show()
```

Case Study 1: Medical X-Ray Enhancement (Negative)
Radiologists often examine X-rays where bones and implants appear dark on a bright background. Inverting (negative transformation) such images makes bones and structures pop out, especially for detecting fractures, foreign objects, or implants.

[4]:
```
# Case Study 1: Medical X-Ray Enhancement (Negative)
img = cv2.imread("D:/23108111/H14.jpg", cv2.IMREAD_GRAYSCALE)
negative = 255 - img
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original X-ray")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(negative, cmap='gray')
plt.title("Negative X-ray")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Case Study 2: Astronomical Image Enhancement (Log Transform)
Astronomers analyzing deep space photos often face the challenge of seeing faint stars or galaxies next to very bright ones. Log transformation enhances low-intensity (dim) regions without letting bright areas dominate the image.

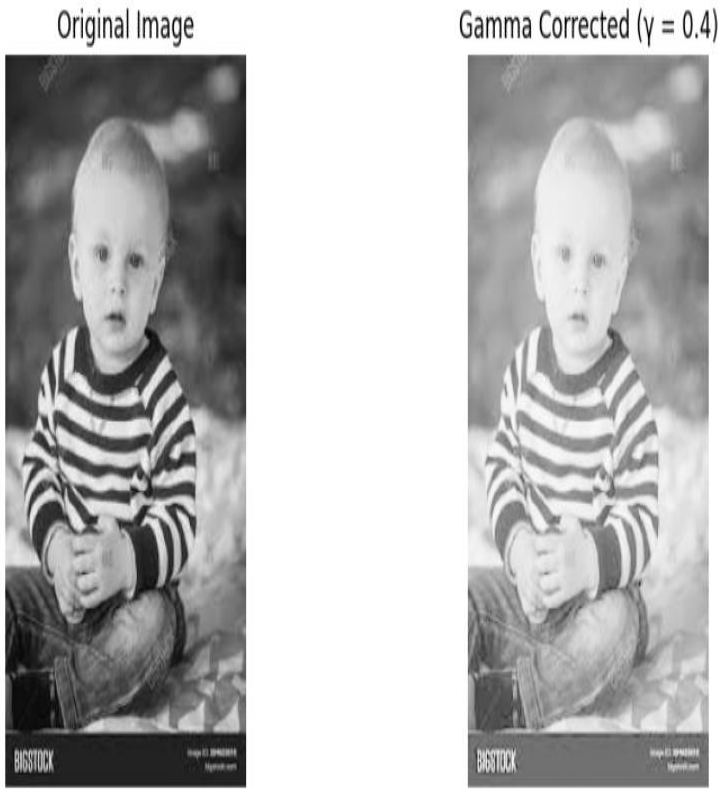
```
# Case Study 2: Astronomical Image Enhancement (Log Transform)
img = cv2.imread("D:/23108111/H15.jpg", cv2.IMREAD_GRAYSCALE)
img_float = np.float32(img)
log_transformed = cv2.normalize(np.log1p(img_float), None, 0, 255, cv2.NORM_MINMAX)
log_transformed = np.uint8(log_transformed)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Astronomical Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(log_transformed, cmap='gray')
plt.title("Log Transformed Image")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Case Study 3: Gamma Correction in Photography

Digital cameras capture images in linear intensity space, which doesn't align with how human eyes perceive brightness. Gamma correction simulates human brightness sensitivity and ensures accurate image display on screens.

```
5]: # Case Study 3: Gamma Correction in Photography
img = cv2.imread("D:/23108111/H16.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gamma = 0.4
gamma_corrected = np.array(255 * (gray / 255) ** gamma, dtype='uint8')
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(gamma_corrected, cmap='gray')
plt.title(f"Gamma Corrected (γ = {gamma})")
plt.axis('off')
plt.tight_layout()
plt.show()
```



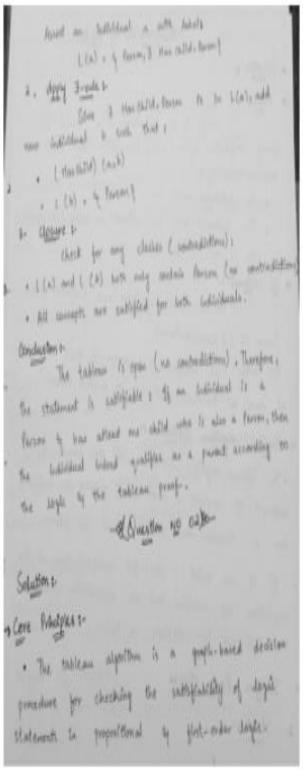
Case Study 4: License Plate Enhancement in Low-Light Surveillance
Surveillance cameras often capture low-light or poorly illuminated images at night.
Important details like license plates appear too dark or faded, making recognition difficult.
Goal: Enhance low-light images using contrast stretching to make license plates readable.

```
# Case Study 4: License Plate Enhancement in Low-Light Surveillance
img = cv2.imread("D:/23108111/H9.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
min_val = np.min(gray)
max_val = np.max(gray)
stretched = ((gray - min_val) / (max_val - min_val) * 255).astype('uint8')
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Low-Light Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(stretched, cmap='gray')
plt.title("Contrast Stretched")
plt.axis('off')
plt.tight_layout()
plt.show()
```

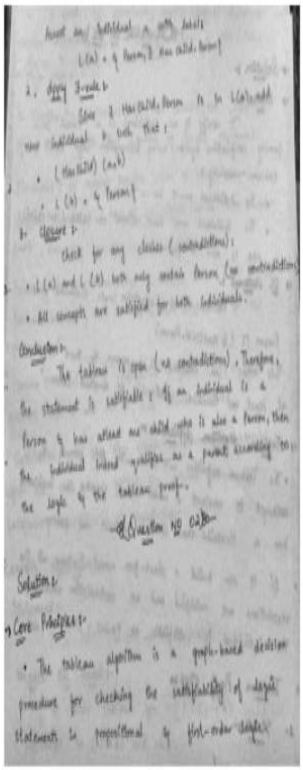
Case Study 5: Old Document Enhancement for OCR
Old scanned documents (e.g., historical records or faded forms) often have uneven lighting or faded ink, making it hard for OCR (Optical Character Recognition) to extract text.
Goal: Enhance contrast using CLAHE (Contrast Limited Adaptive Histogram Equalization) for better readability and OCR accuracy.

```
# Case Study 5: Old Document Enhancement for OCR
img = cv2.imread("D:/23108111/HH5.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
clahe = cv2.createCLAHE(clipLimit=4.0, tileGridSize=(8, 8))
enhanced = clahe.apply(gray)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Document")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(enhanced, cmap='gray')
plt.title("CLAHE Enhanced")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Original Document



CLAHE Enhanced



Case Study 1: Enhancing Night Surveillance Footage Using Image Negative
A security agency provides black-and-white surveillance images taken under poor lighting. Bright lights (e.g., street lamps) dominate the scene, while human figures appear as dark silhouettes.
Task
Use image negative transformation to:

- **Invert the image intensities.**
- **Bring out hidden features in darker areas (e.g., faces or vehicle plates).**

9]:
```
# Case Study 1: Enhancing Night Surveillance Footage Using Image Negative
img = cv2.imread("D:/23108111/H9.jpg", cv2.IMREAD_GRAYSCALE)
negative = 255 - img
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(negative, cmap='gray')
plt.title("Negative Image")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Case Study 2: Contrast Stretching for Foggy Road Scenes in Self-Driving Cars
A self-driving car system struggles to identify road lanes and obstacles in foggy or low-contrast conditions.
Task
Apply contrast stretching to:

- **Enhance the difference between road and non-road areas.**
- **Improve lane detection under poor visibility.**

```
# Case Study 2: Contrast Stretching for Foggy Road Scenes in Self-Driving Cars
img = cv2.imread("D:/23108111/H10.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
min_val = np.min(gray)
max_val = np.max(gray)
stretched = ((gray - min_val) / (max_val - min_val) * 255).astype('uint8')
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Low-Light Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(stretched, cmap='gray')
plt.title("Contrast Stretched")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Case Study 3: Logarithmic Enhancement for Satellite Imagery

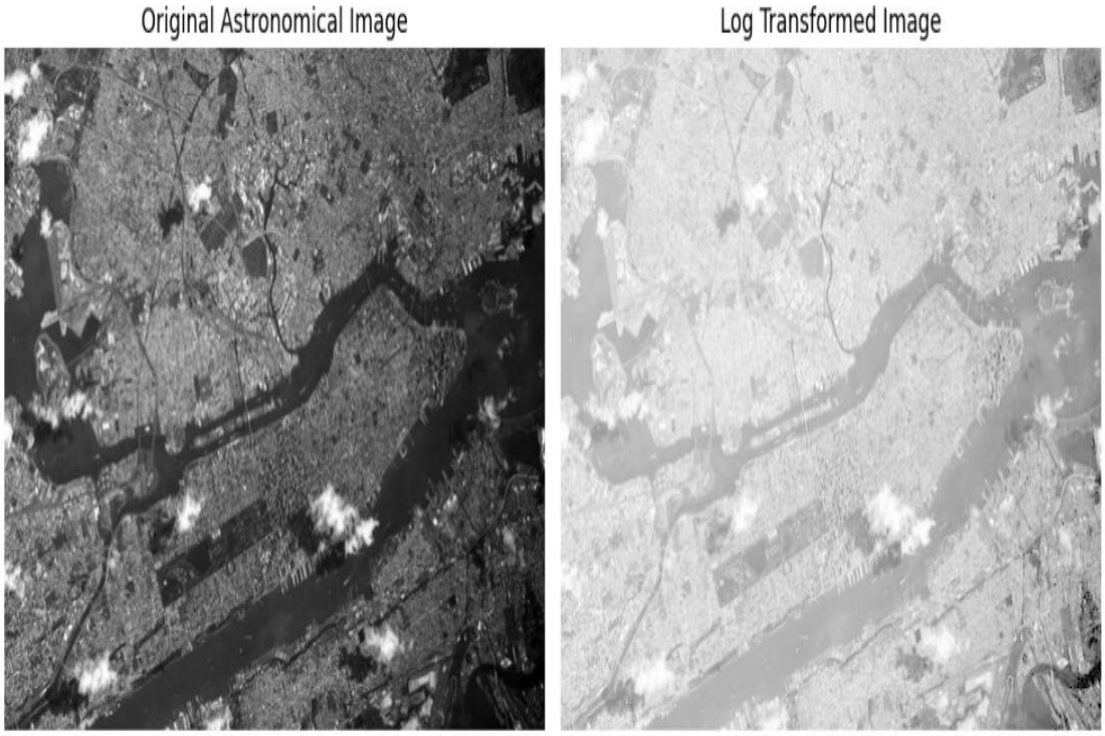
In satellite images, urban structures and roads are faint compared to natural features like forests or deserts. The raw images have low detail in darker pixel areas.

Task

Use log transformation to:

- **Reveal subtle infrastructure in satellite images.**
- **Enhance weak reflectance areas.**

```
# Case Study 3: Logarithmic Enhancement for Satellite Imagery
img = cv2.imread("D:/23108111/H7.jpg", cv2.IMREAD_GRAYSCALE)
img_float = np.float32(img)
log_transformed = cv2.normalize(np.log1p(img_float), None, 0, 255, cv2.NORM_MINMAX)
log_transformed = np.uint8(log_transformed)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Astronomical Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(log_transformed, cmap='gray')
plt.title("Log Transformed Image")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Case Study 4: Gamma Correction on Medical Thermal Imaging

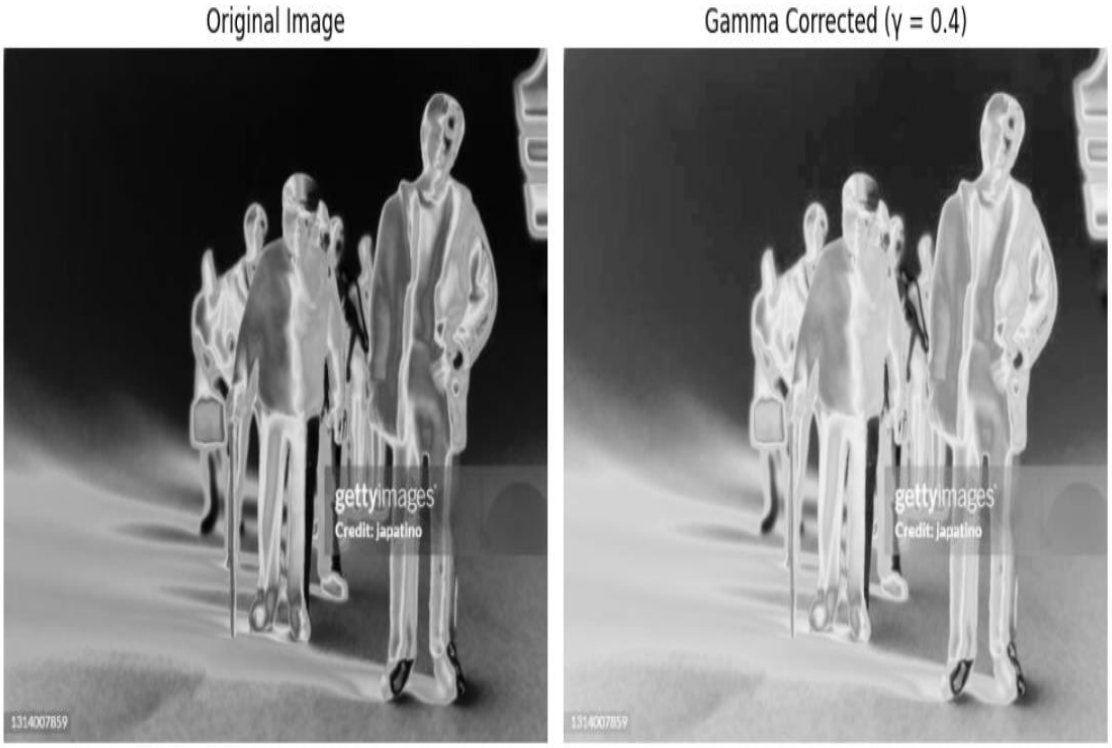
A hospital uses thermal imaging to detect inflammation. Some images are underexposed, and crucial hot zones are barely visible.

Task

Use gamma correction with γ < 1 to:

- **Brighten the image.**
- **Emphasize subtle temperature differences.**

```
]: # Case Study 4: Gamma Correction on Medical Thermal Imaging
img = cv2.imread("D:/23108111/H13.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gamma = 0.4
gamma_corrected = np.array(255 * (gray / 255) ** gamma, dtype='uint8')
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(gamma_corrected, cmap='gray')
plt.title(f"Gamma Corrected (γ = {gamma})")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Case Study 5: Intensity Slicing for Agricultural Disease Detection

Scenario

In drone-captured grayscale images of crop fields, disease-affected regions appear within a narrow intensity band.

Task

Use intensity slicing to:

- **Highlight specific intensity ranges that indicate infected crops.**
- **Suppress all other intensity levels.**

```
# Case Study 5: Intensity Slicing for Agricultural Disease Detection
img = cv2.imread("D:/23108111/H17.jpg",cv2.IMREAD_GRAYSCALE)
lower = 90
upper = 140
sliced = np.zeros_like(img)
sliced[(img >= lower) & (img <= upper)] = 255
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original Drone Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(sliced, cmap='gray')
plt.title("Disease Highlighted (Intensity Slicing)")
plt.axis('off')
plt.tight_layout()
plt.show()
```

https://github.com/Habiba-Bibi/